SuperChat — Technical Report

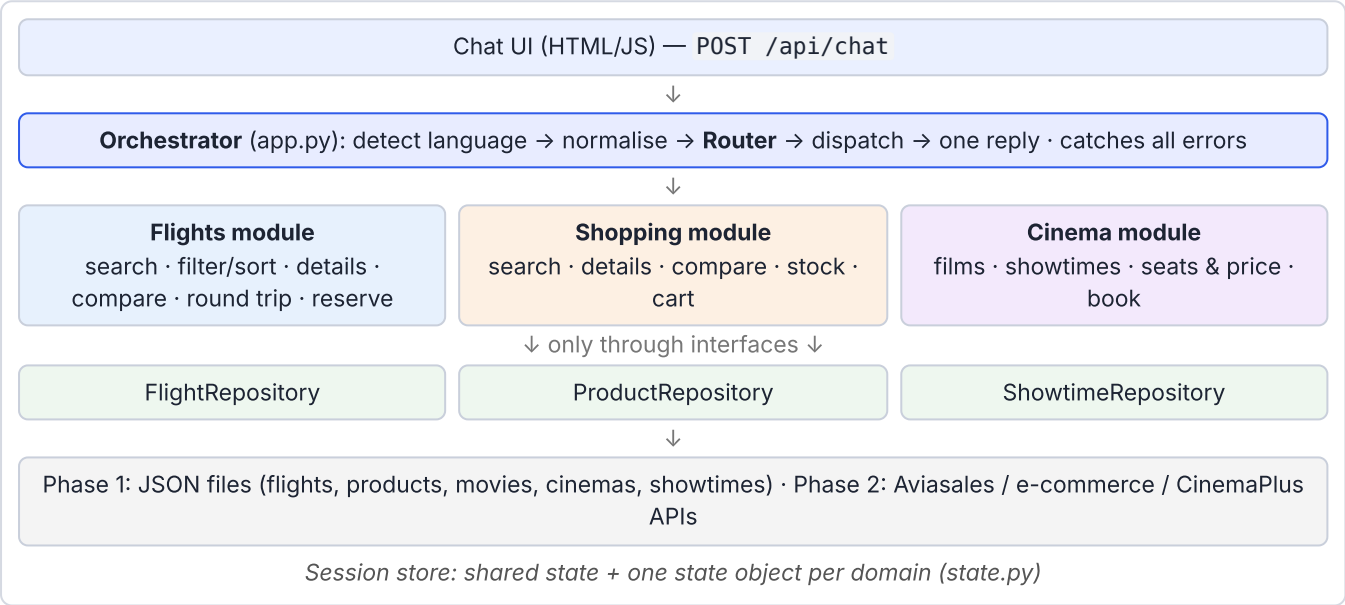
One conversational assistant for Flights, Shopping and Cinema · Phase 1 (mock data)

Elvina Mustafayeva

1. Overview

SuperChat combines the Aviasales, Shopping Assistant and CinemaPlus experiences in one chat window. Users never choose a mode. Each message is classified as *flights, shopping, cinema, mixed, follow-up, ambiguous* or *out of scope*, sent to the right module(s), and answered in the user's language (English, Azerbaijani or Russian). The system runs offline on the official mock dataset with one command (`python run.py`). It uses only the Python standard library, so it has no dependencies, needs no API keys and gives repeatable results.

2. Architecture



Supporting components: `nlp.py` handles normalisation, language detection, dates, quantities, ordinals and price ranges. `entities.py` holds an index of cities, airlines, flight numbers, brands, product names, film titles and cinemas, **built from the data at start-up**. `lexicon.py` holds the multilingual cue lists.

3. Routing method

Approach: hybrid, rule-based and data-driven. Each message is normalised: lower-cased, with Azerbaijani letters folded to ASCII (ə→e, ş→s, ı→i…), so "uçuş" and "ucus" match. The message is then scored per domain with two kinds of evidence:

- **Strong cues**: topic words in three languages (*flight / uçuş / рейс*, *laptop / noutbuk / ноутбук*, *film / kino / фильм*) and **entities from the data** (cities with Azerbaijani case endings such as *Bakıdan*, *İstanbula*; airline names; flight numbers; brands; product names; film titles; cinema names). Film titles are matched first and masked, so "Spring in **Baku**" is not read as a city.
- **Weak cues**: attributes typical of follow-ups (*direct, baggage, in stock, add, hall*).

The decision then uses these cues together with the session state:

Condition	Category
Strong cues for two or more domains	**mixed**: the text is split into clauses (and / və / и / commas); each clause goes to its domain; the replies are joined into one message

Condition	Category
Strong cues for one domain, and the message has an anaphora marker (“the cheaper one”, “it”, “back to”, ordinals, “bəs”, “а на…”) and that domain already has state	**follow-up**
Strong cues for one domain	that **domain**
An out-of-scope cue (weather, hotel, joke, погода…) with no topic word	**out of scope**, even if a city is mentioned (“hotel in Istanbul”)
Only weak cues, or a contextual message (booking words, a date, a quantity, an ordinal)	**follow-up** to the last active domain, or the only domain with state; otherwise **ambiguous**
Vague request (“something good”, “recommend”, “book it”) with no context	**ambiguous**: one short clarifying question
Nothing matched	**out of scope**: polite refusal plus what SuperChat can do

After a clarifying question, the next short answer (“the film”) is merged with the original message (“Book two seats”). The intent and the quantity are kept, so the booking carries on.

Why this approach, and the trade-offs. Rules give *deterministic, explainable* decisions: the UI and the test log show exactly which cues fired. They also need no network, react in under a millisecond, cost nothing, and are easy to debug live in a demo. Because entities come from the data, a new city or film is recognised without code changes. The cost is **recall on unseen phrasing**: typos and indirect requests are missed (see §6). An LLM with function calling would handle that better, but it adds latency, cost, non-determinism and a network dependency. `Router` is a Python `Protocol`, so an LLM or embedding classifier can be added later as a *fallback* used only when the rules find nothing. That would keep the fast, predictable path for the common cases.

4. Conversation state

Each session holds a **shared state** and **three independent domain states**:

State	Contents
Shared	language of the last message; last active domain; preferences stated so far (number of passengers or tickets); pending clarifying question; full transcript
Flights	current query (origin, destination, date, return date); active filters (direct, airline, time of day, max price, sort); result list; selected flight; bookings
Shopping	current query (type, brand, colour, price range, sort); result list; selected product; **cart**; history of earlier searches with their selections
Cinema	listed films; listed showtimes; which list was shown last; selected film and showtime; booking in progress; bookings

Because domain states never overwrite each other, a user can go flights → cinema → shopping and back, and find each topic as they left it. “Back to the headphones” restores the earlier list *and* the selected item. Follow-ups are resolved against the domain’s last list: an ordinal refers to the most recently shown list (films or showtimes), and “the cheaper one” ranks that list by price. When a reference matches more than one item (“the sci-fi one” when two sci-fi films are on tonight), SuperChat shows both options instead of guessing. Sessions live in an in-memory `SessionStore`; in Phase 2 it can be replaced by Redis behind the same two methods.

5. Data layer design

The modules never read files. They call three abstract interfaces whose method names follow user actions: `FlightRepository.search / get / reserve / cities / airlines`, `ProductRepository.all / get`, and `ShowtimeRepository.movies / cinemas / showtimes / get_showtime / reserve`. The Phase 1 classes

(`Json…Repository`) load the official files. A single factory, `build_repositories()`, decides which implementation to use, so moving to the real APIs means adding new classes there, with authentication, rate limiting, pagination, caching and logging inside them. Failures surface as `DataSourceError`; the orchestrator turns any exception into a friendly message, so the chat never shows a stack trace. Mock reservations check seat availability, lower it in memory and return a reference (`F-1001`, `T-1001`). No payment or personal data is involved.

No invented data. Every reply is built from repository records. When nothing matches, SuperChat says so and lists real alternatives from the data: other dates on that route, the closest products in the category, or other days a film is showing. Questions about cities that are not in the data ("flights to Paris") get the list of supported cities, never made-up results. The test log checks this explicitly.

6. Phase 2: real API integration (implemented ahead of schedule)

`api_repositories.py` provides HTTP versions of all three repositories. They are selected per domain through environment variables, so mock and real sources can be mixed during migration. They are built on a standard-library `HttpClient` that covers the brief's Phase 2 list:

Concern	How it is handled
Authentication	Bearer key per service; Aviasales `X-Access-Token`; tokens redacted from logs
Rate limits	client-side minimum interval; 429 retried, honouring `Retry-After`
Pagination	`paginate()` follows `next_page` until all items are read
Latency and failures	timeouts; up to 3 retries with exponential back-off on 5xx and network errors; a final failure gives a polite reply
Caching	TTL cache per source (products 60 s … showtimes 15 s), cleared after a reservation so seat counts stay fresh
Safe reservations	every POST carries an `Idempotency-Key`, so a retried booking is never made twice
Logging	method, URL, status and latency for every call

Aviasales is connected through its public Data API (`/aviasales/v3/prices_for_dates`). That API returns cached fares from the last 48 hours and has no arrival time, seat count, baggage or booking. SuperChat shows these fields as unknown and replaces "reserve" with a hand-over link to Aviasales. The organisers' APIs follow `docs/API_CONTRACT.md`. `tools/fake_api.py` serves that contract from the mock data with auth, paging, 409s and random 503/429 errors. In tests, **all conversations give the same replies on the fake API as on the JSON files, even with 25% of calls failing**.

7. Testing and results (details in TEST_LOG.md)

Test set	Size	Accuracy
Development (used for tuning)	70	100%
Held-out (written separately, not tuned on)	48	100%
Stress: typos, no diacritics, indirect phrasing (not tuned on)	30	73%
End-to-end conversation turns (the 8 brief scenarios + 4 extra, incl. AZ/RU)	30	30/30 pass
Phase 2 API mode: parity, retries, paging, idempotency, failures, Aviasales mapping	11	11/11 pass

The test sets cover all seven categories in all three languages, and the stress set is the most realistic estimate. Its errors come from typos ("headphnes"), intents without a product noun ("something for running"), vague

phrases missing from the cue list, and one domain word inside an unrelated question (“traffic to the airport”). In every error the system fails *safely*: it asks a question or declines, and never answers with wrong-domain data.

8. Known limitations and next steps

- **Recall on unseen wording.** Typos and paraphrases fall outside the cue lists. Next steps: fuzzy matching, plus an LLM or embedding fallback for messages where the rules find no cue.
- **Reply wording is template-based.** Replies are clear and consistent, but less natural than generated text. Film and product descriptions stay in the language of the dataset.
- **Round trips** show outbound and return lists; only the outbound flight can be selected and reserved.
- **Mixed messages** are split on conjunctions. A sentence that mixes domains without a conjunction is still routed to every domain, but each module reads the whole sentence.
- **State lives in memory** and resets when the server restarts. In mock mode, bookings are also in memory and shared across sessions, as a real inventory would be.
- **Real organisers’ APIs** are not public yet. The connectors follow an assumed contract (same field names as the mock data) and may need a small field mapping once the documentation arrives. Aviasales needs a personal Travelpayouts token.
- **Bonus tasks (all three implemented in the web version):** multilingual EN/AZ/RU; hands-free voice input (Web Speech API, automatic start/end-of-speech detection, optional spoken replies; Chrome/Edge only); and visual search (MobileNetV2 in the browser, states its confidence, asks when unsure). ImageNet has no headphones class, so headphone photos always lead to a question.

Sources: Python 3 standard library; web bonus uses onnxruntime-web 1.19.2 (MIT) and MobileNetV2 from the ONNX Model Zoo (Apache-2.0). Mock dataset supplied by the organisers.